

PIANO NUMÉRIQUE

Conception d'un clavier à une octave

Widad NORI • Élie CORBIÈRE • Julien FAUVEL

Léo ROUPENEL • Wissem M'TIR • Claire MESTRE PINCHO

Encadré par M. Nicolas SIMOND | ENSEA - Projet d'Électronique 1A

Context & Introduction

The initial objective of this project was to design an interactive and functional system from scratch, utilizing our skills in analog electronics, digital electronics, and signal processing.

Our ambition was to create a four-octave keyboard. However, design rules and constraints related to the **maximum dimensions allowed** for printed circuit board (PCB) manufacturing led us to reduce its size.

We therefore opted for a complete, functional, and optimized instrument: **a single-octave digital piano.**



Project Specifications (SysML)



Functional Requirements

Detect 13 notes (C to C).
Management of **polyphony** to play several notes simultaneously. Mixed sound output on Speaker or Jack.



Performance

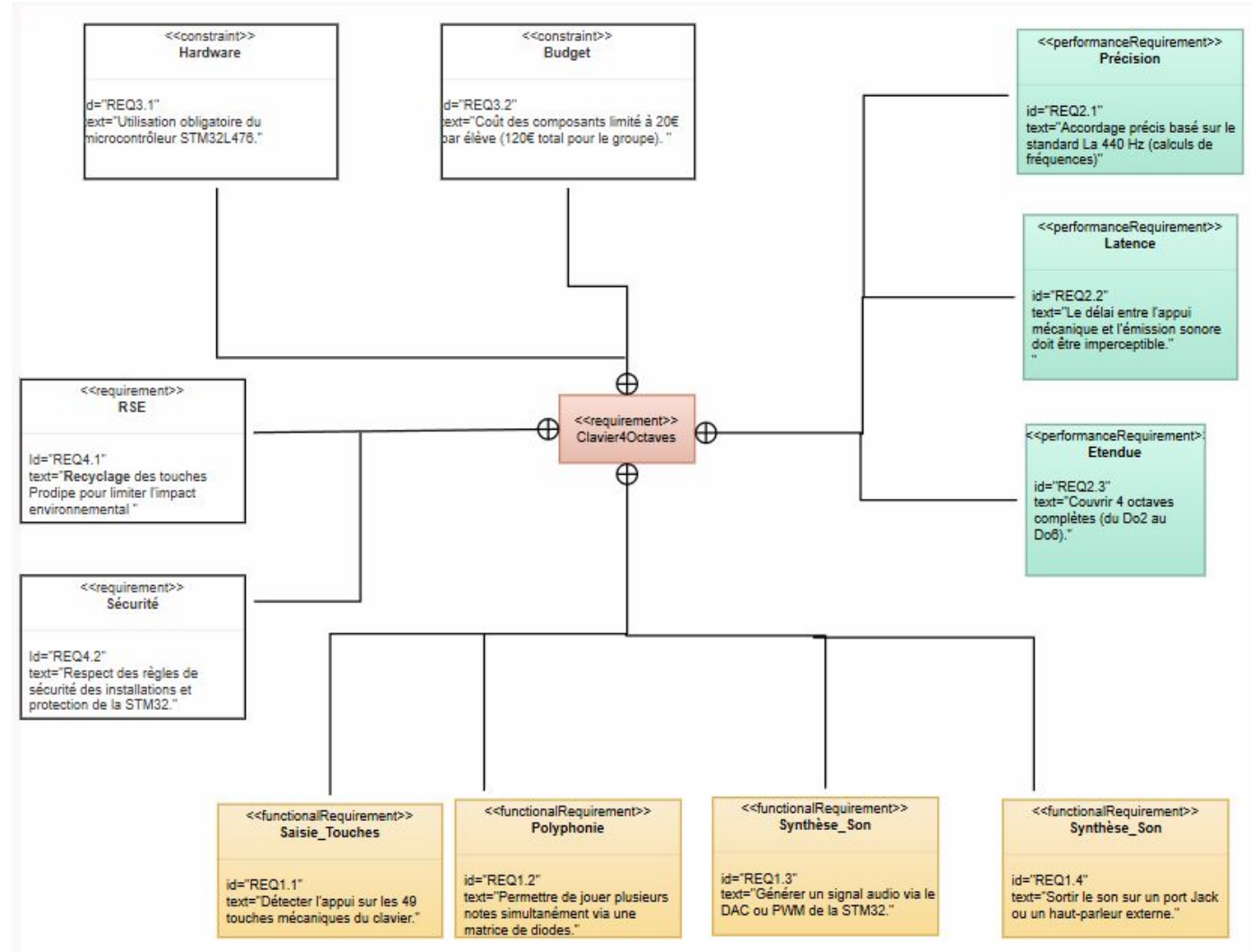
Precise tuning based on the **A 440 Hz** standard calculated mathematically. The latency between mechanical key press and sound emission must be imperceptible.



Hardware & Budget

Mandatory use of the **Nucleo STM32L476** board. Cost strictly limited to €20 per student (€120 max for the whole group).

Diagramme SysML



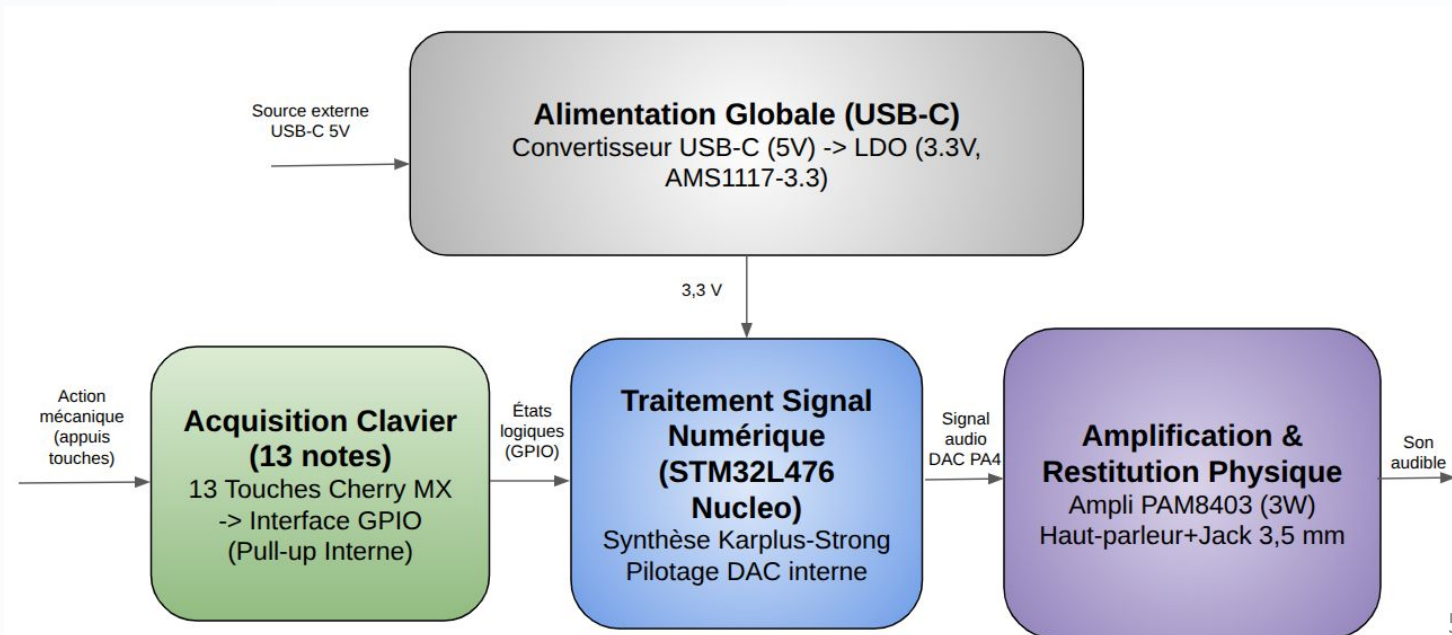
System Architecture

1. Power Supply: USB-C (5V) secured by resistors on CC1/CC2, then regulated by an AMS1117 to provide a clean 3.3V to the microcontroller.

2. Acquisition: 13 keys connected directly to the GPIO pins of the microcontroller (low state detection via internal pull-up).

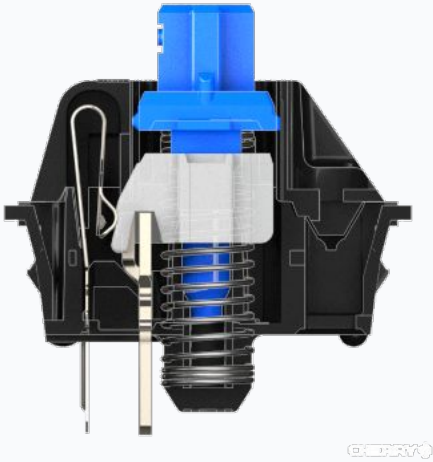
3. Processing (STM32): The heart of the system. It scans the GPIOs and generates the 12-bit sound signal via its internal DAC on pin PA4.

4. Audio Output: The analog signal is amplified and distributed to the speaker or the headphone output.



Key Components

Acquisition: Cherry MX Switches (Silent Black)



Reliability: 50 million keystrokes

Comfort: Integrated dampeners to avoid audio pollution

Wiring: Using one GPIO pin per key ensures polyphony.

Amplification: PAM8403

Audio amplifier powered by 5V

Necessity: The microcontroller signal is too weak for a speaker.

Power: Delivers up to 3 Watts to the speaker.

Efficiency (Class D): Very efficient (90%), converts energy into sound without heating.

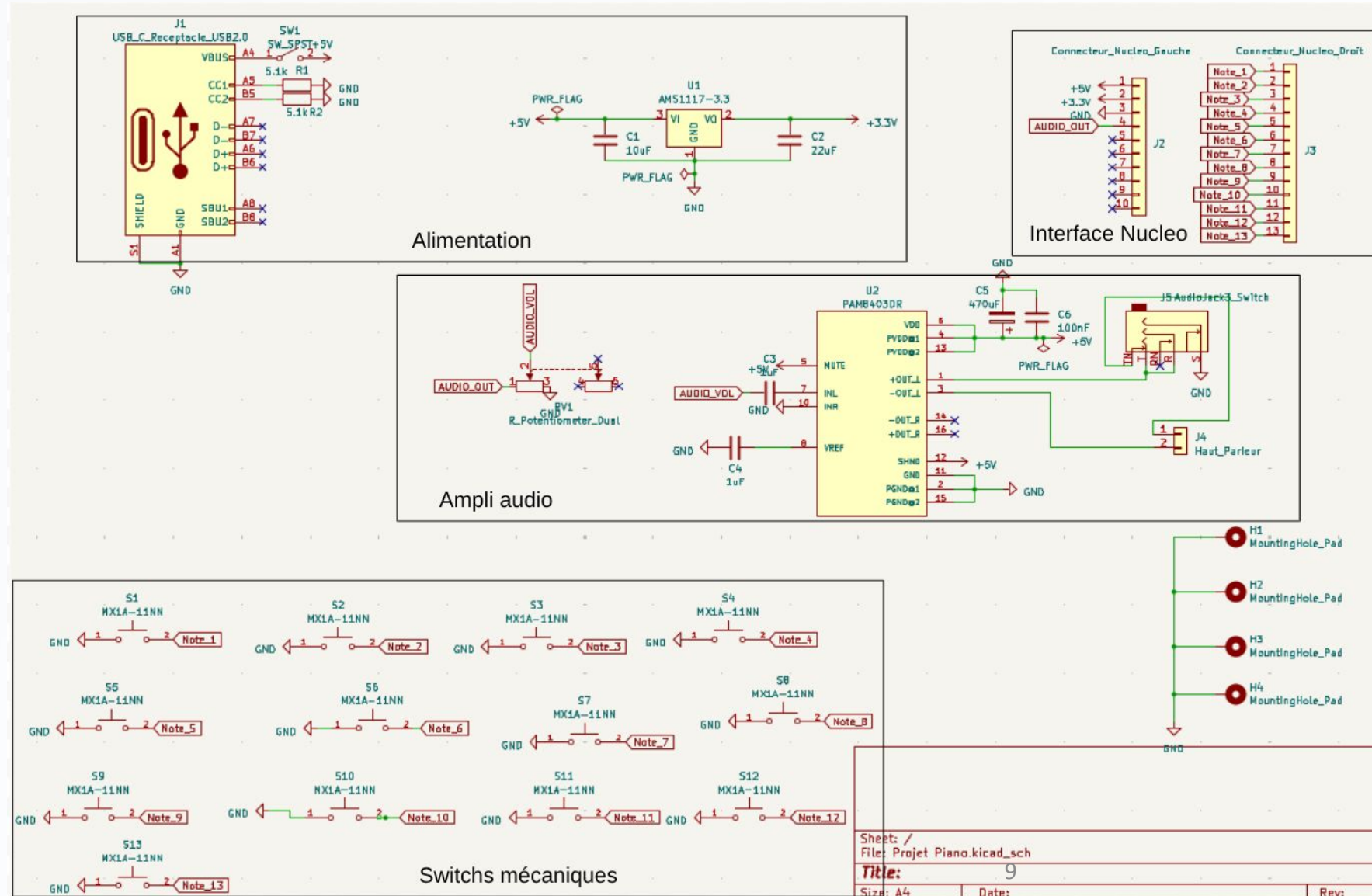
List of components

Block	Component	Reference / Specs	Role
Input	Mechanical Switches (x13)	Cherry MX Silent Black	Reliability (50M keystrokes) & Silence
Processing	Microcontroller	Nucleo-L476RG (STM32)	Karplus-Strong Synthesis & DAC
Power	USB-C Connector	USB4105-GF-A (PD Ready)	Universal 5V Input (CC1/CC2 5.1k Ω)
Power	LDO Regulator	AMS1117-3.3 (SOT-223)	Stable 3.3V Logic Supply
Output	Audio Amplifier	PAM8403 (Class D, 3W)	High efficiency, low heat dissipation
Output	Potentiometer	Dual RK16312A (10 k Ω)	Clean analog volume control
Output	Speaker / Jack Socket	Digikey 1528-2752 / FC6813	Mixed output with hardware cutoff

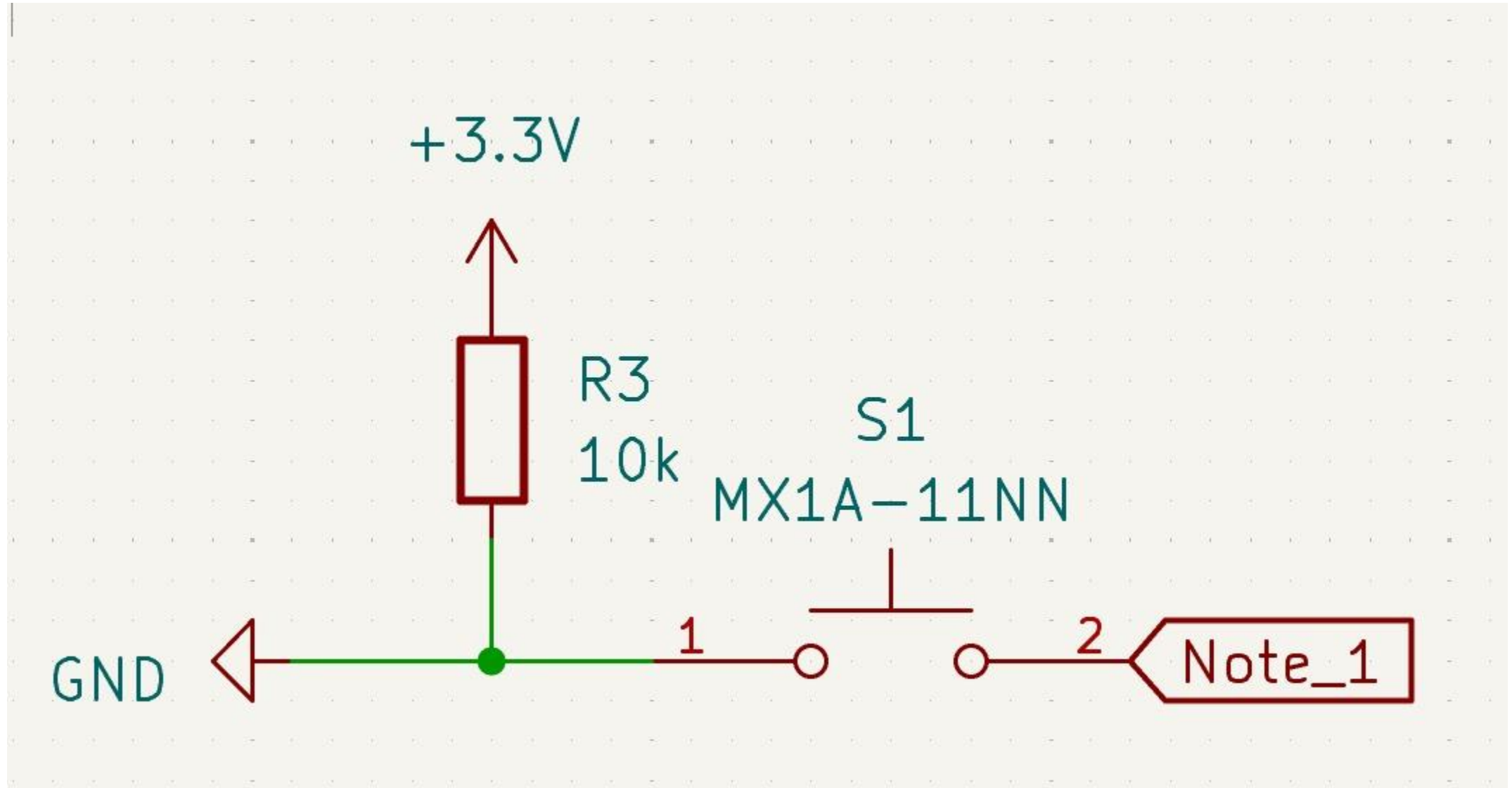
Conception Électronique

Schémas, Routage et Réalisation du PCB sous KiCad

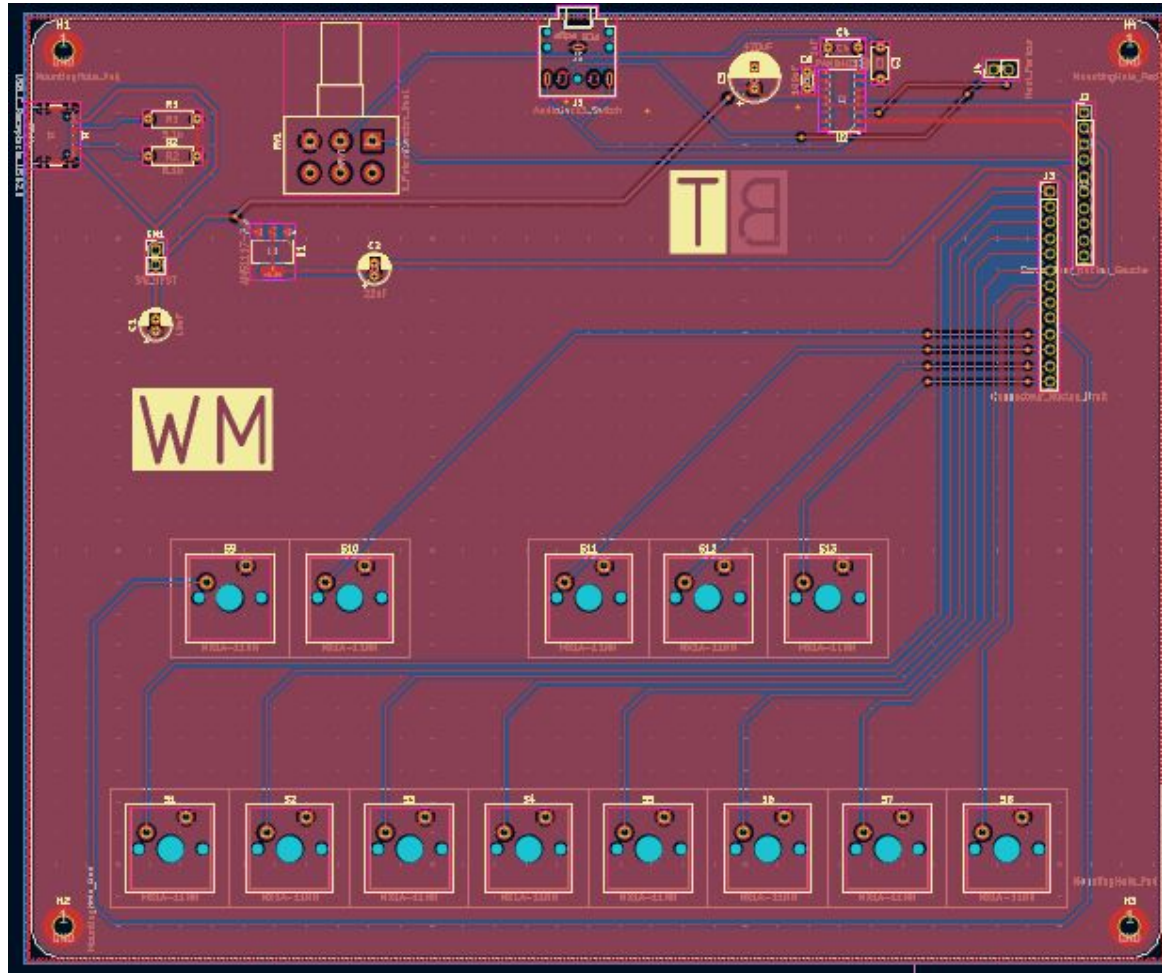
CAO sous KiCad



CONCEPTION DU PCB



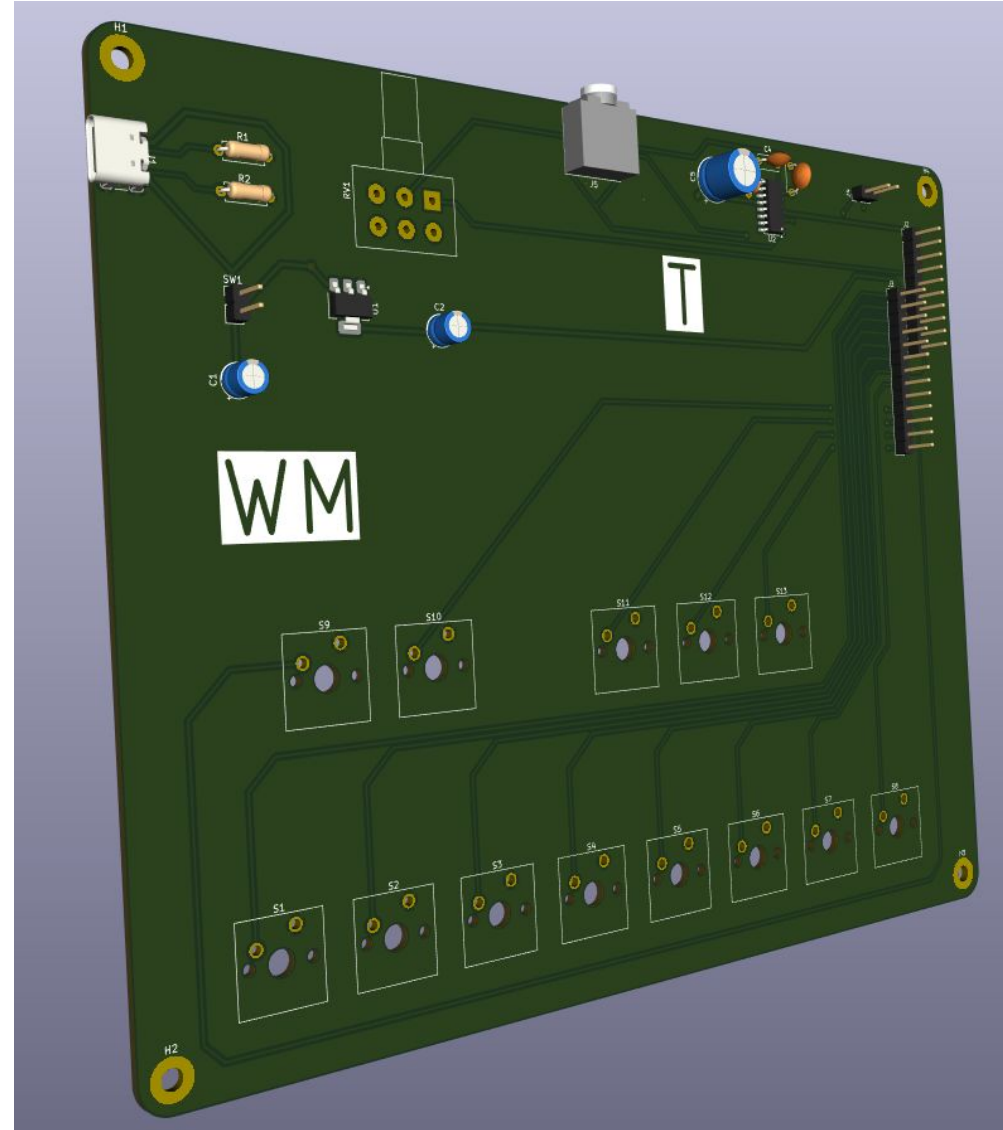
CONCEPTION DU PCB



Routeage 2 Couches

Pistes logiques au dos pour préserver le signal analogique de l'amplificateur en façade.

CONCEPTION DU PCB



Vue 3D du PCB (KiCad)

Printed Circuit Board Details



"Shield" Format: The circuit was designed as an expansion board fitting directly onto the Nucleo via headers. This protects the microcontroller from permanent soldering.



Full Ground Plane: A copper fill connected to GND (zone fill) on both sides forms an essential electromagnetic shield to avoid background noise (hiss) in the audio signal.



Playing Ergonomics: The 13 Cherry MX footprints are precisely arranged to reproduce the standard spacing of a piano octave (aligned white keys, offset black keys).



USB-C Security: Critical addition of two 5.1k Ω resistors on pins CC1/CC2 to force Power Delivery chargers to deliver only 5V (circuit protection).

Traitement du Signal

Synthèse Sonore & Implémentation

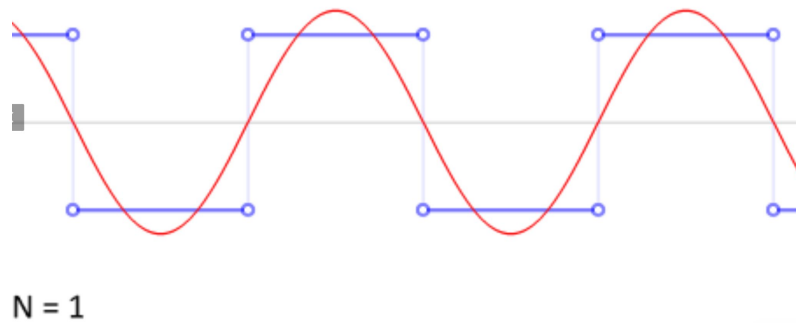
Logicielle

Algorithm Choice

To synthesize the piano sound, we had to choose between several methods:

- **Additive synthesis:** We add elementary signals to simulate a piano sound
- **FM synthesis:** Method using frequency modulation to synthesize a sound
- **Karplus-Strong algorithm:** A white noise loop is played back on itself and filtered with a low-pass filter

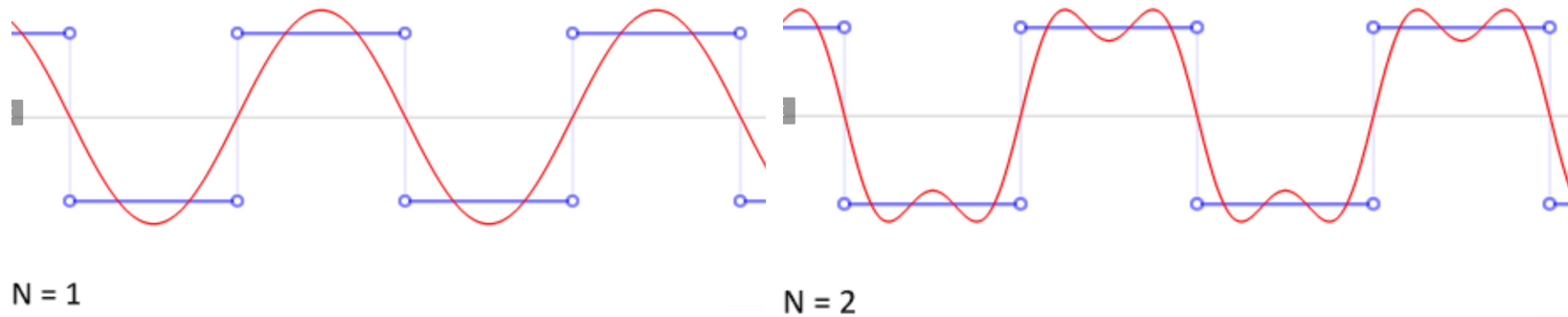
Additive synthesis



Pros and cons :

- **Simple to implement:** Very simple method
- **Rich sound:** We directly control the amount of harmonics, the result can be rich and easily adjustable
- **Very heavy :** If we want to synthesize a high-quality sound, we need a large amount of harmonics, which takes a lot of calculation time
- **Originality :** method with little interest

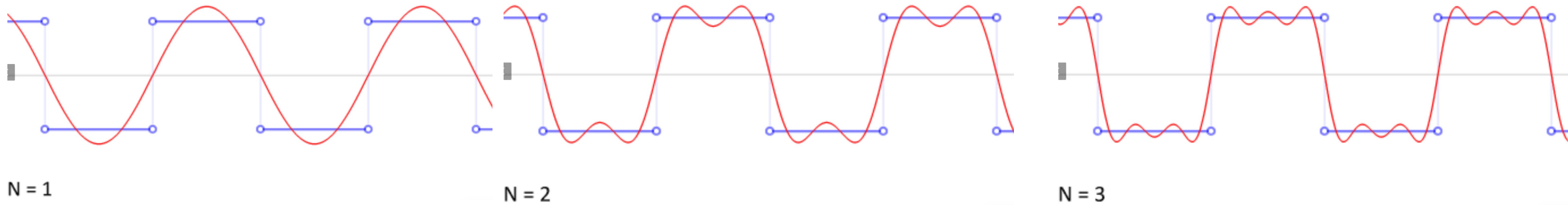
Additive synthesis



Pros and cons :

- **Simple to implement:** Very simple method
- **Rich sound:** We directly control the amount of harmonics, the result can be rich and easily adjustable
- **Very heavy :** If we want to synthesize a high-quality sound, we need a large amount of harmonics, which takes a lot of calculation time
- **Originality :** method with little interest

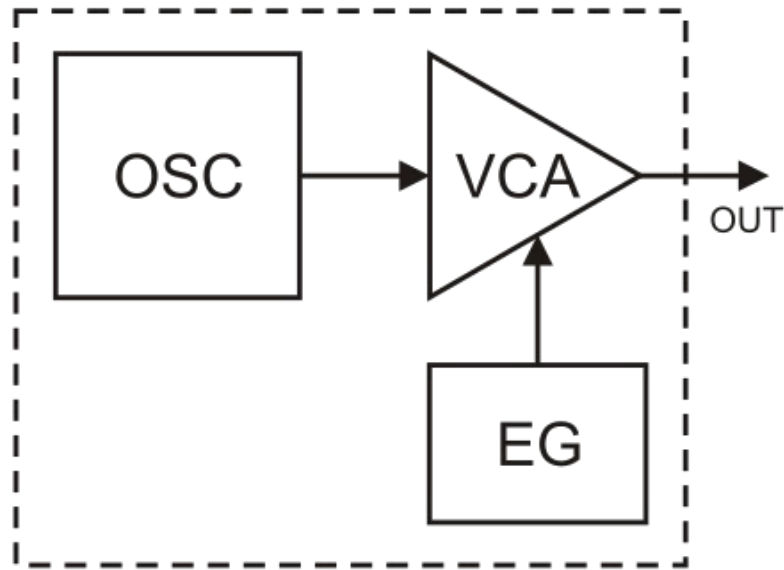
Additive synthesis



Pros and cons :

- **Simple to implement:** Very simple method
- **Rich sound:** We directly control the amount of harmonics, the result can be rich and easily adjustable
- **Very heavy :** If we want to synthesize a high-quality sound, we need a large amount of harmonics, which takes a lot of calculation time
- **Originality :** method with little interest

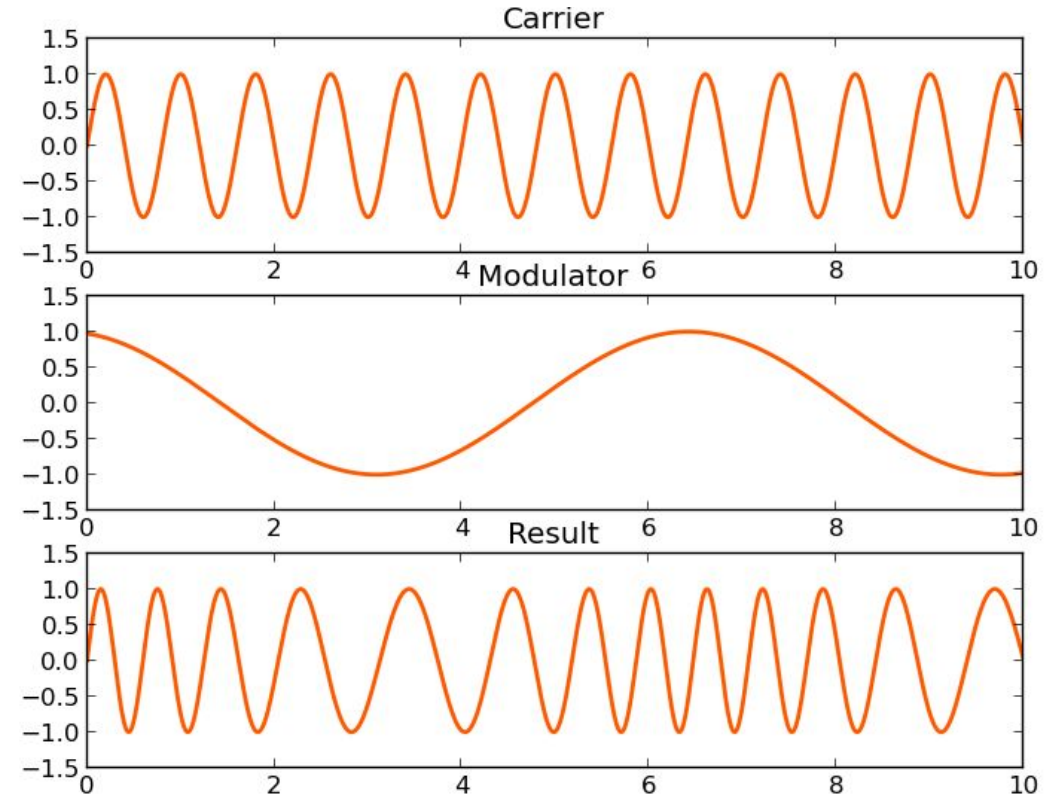
FM Synthesis (frequency modulation)



OSC: Oscillator

EG: Envelope generator

VCA: Voltage controlled amplifier



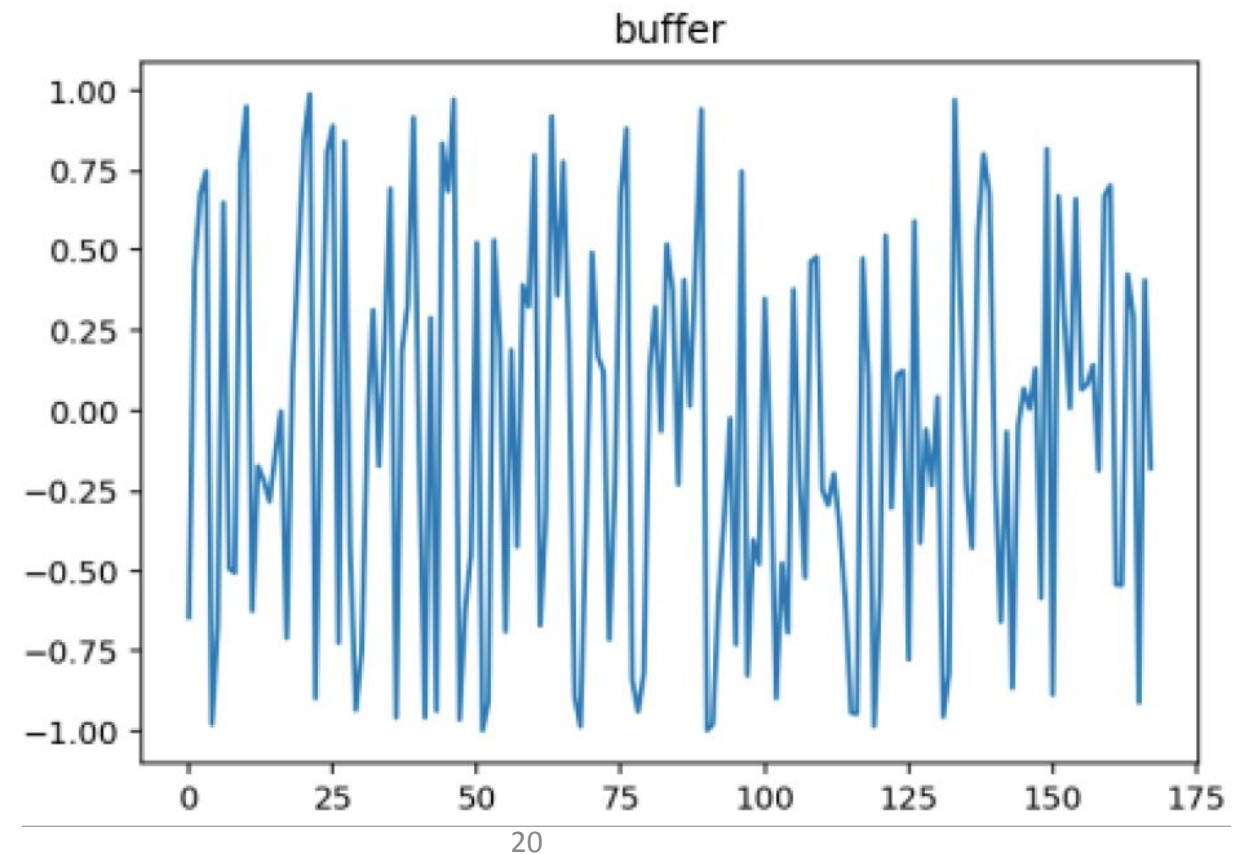
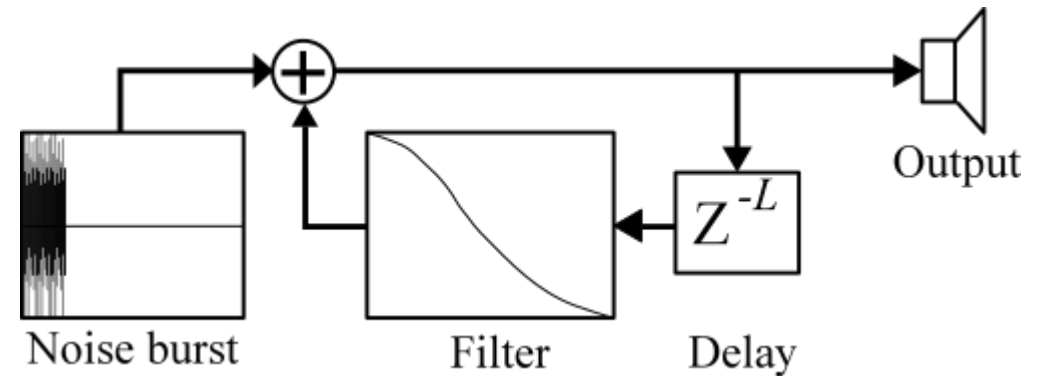
Pros and cons:

- **Rich and natural sound:** You can directly control the amount of harmonics
- **Very sensitive and complicated to adjust:** a slight modification of the carrier frequency or the modulation frequency can completely change the timbre

The Karplus-Strong Algorithm

To generate sound without pre-recorded samples, we mathematically model a struck string:

- **The Impact:** The "hammer" is simulated by a short array filled with white noise (random values).
- **The Resonance:** This array is read in a loop. The size of the array (the delay) defines the pitch of the note.
- **The Damping:** At each loop, an average with the adjacent value acts as a low-pass filter.



```
17  
18 def bruit_blanc(nb_samples):  
19     return np.random.uniform(-1, 1, nb_samples)  
20
```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.io.wavfile import write

import sounddevice as sd

sample_rate = 44100

def bruit_blanc(nb_samples):
    return np.random.uniform(-1, 1, nb_samples)

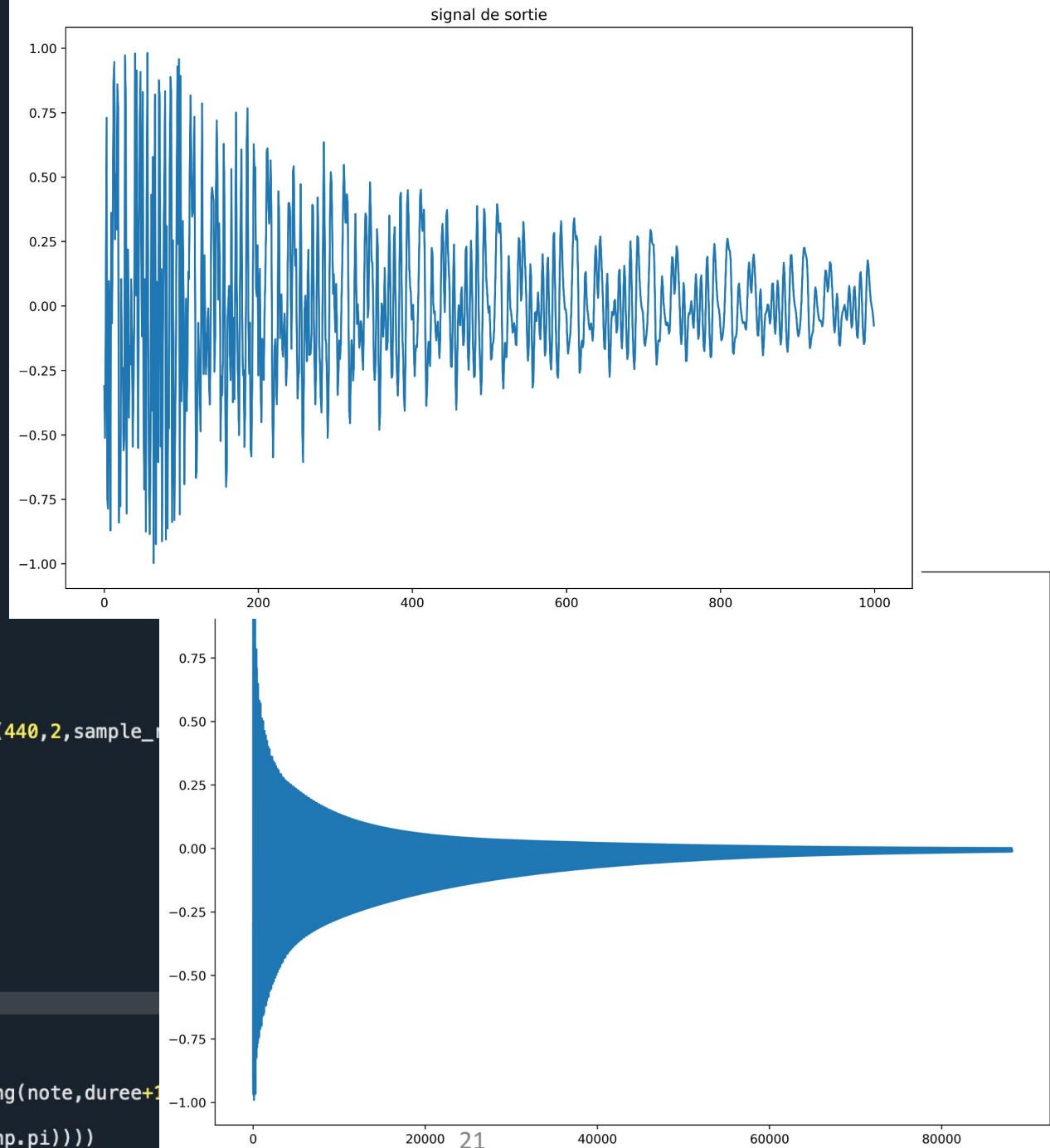
def karplus_strong(frequency, duration, sample_rate=44100, damping=0.5):
    N = int(sample_rate / frequency)
    buffer = bruit_blanc(N)
    print('buffer', buffer)
    plt.plot(buffer)
    plt.title('buffer')
    plt.show()
    output = []
    for i in range(int(duration * sample_rate)):
        current = buffer[0]
        output.append(current)
        avg = damping * 0.5 * (buffer[0] + buffer[1])
        buffer = np.append(buffer[1:], avg)
    plt.plot(output)
    #plt.plot(output[:1000])
    plt.title('signal de sortie')
    plt.show()
    sd.play(output)
    return np.array(output, dtype=np.float32)

write("/Users/julienfauvel/Downloads/karplus_5.wav", sample_rate, karplus_strong(440, 2, sample_rate))

Do = 261.6
Re_b = 277.2
Re = 293.7
Mi_b = 311.1
Mi = 329.6
Fa = 349.2
Sol_b = 370
Sol = 392
La_b = 415.3
La = 440
Si_b = 466.2
Si = 493.9

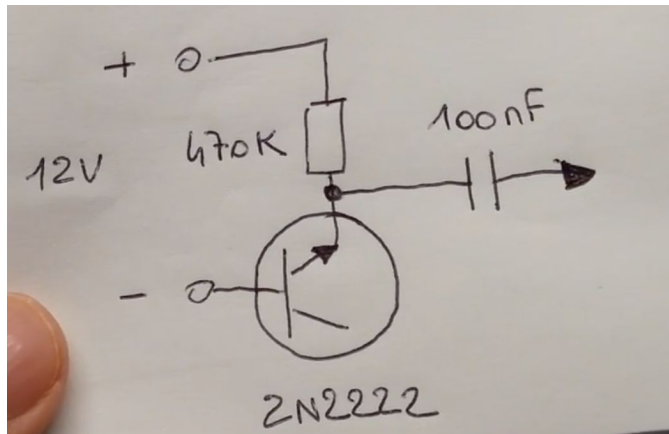
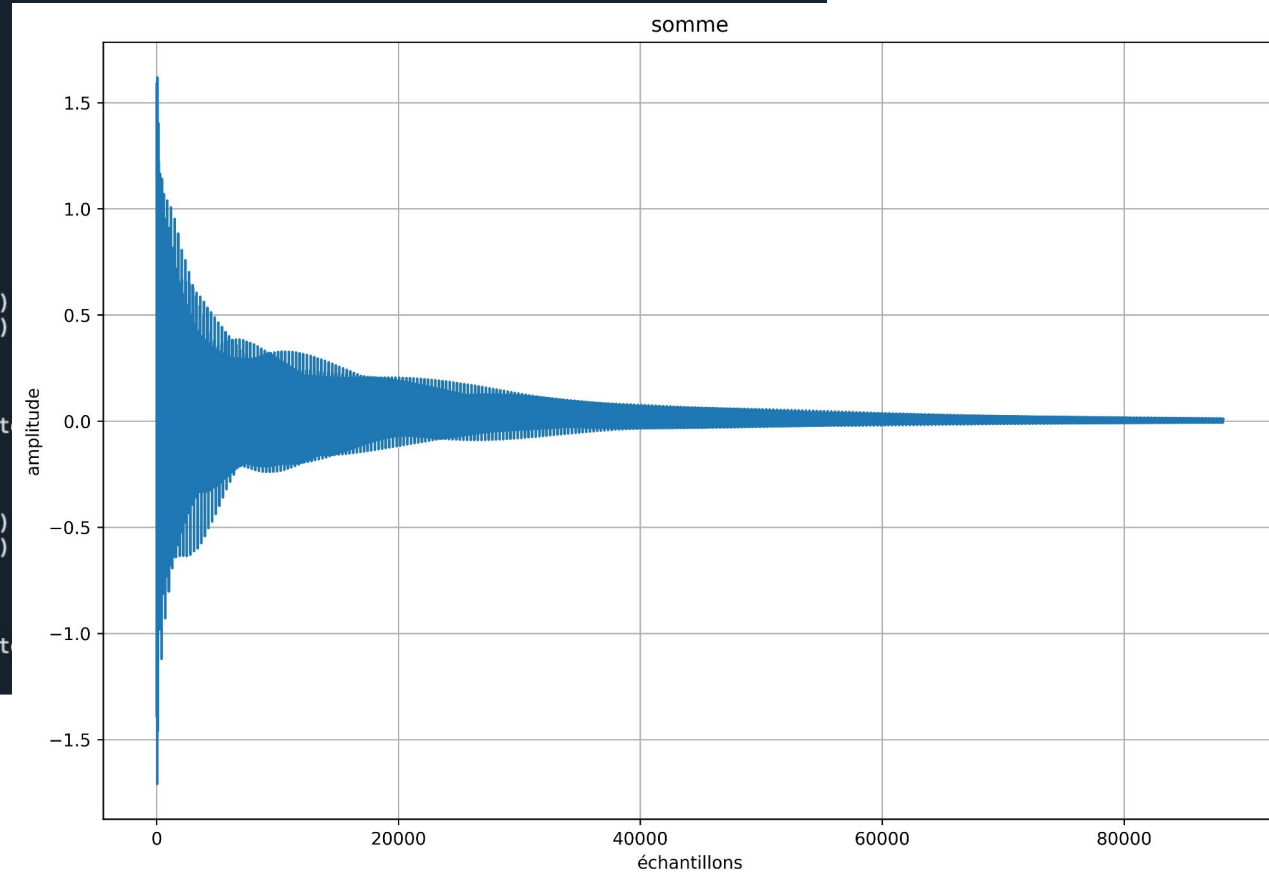
def note_piano(note, duree):
    write("/Users/julienfauvel/Downloads/AAAAA1.wav", sample_rate, karplus_strong(note, duree+1, sample_rate, ((2*np.arctan(140*duree))/(np.pi))))
    print(2*np.arctan(140*duree)/(np.pi))
    return(karplus_strong(note, duree+1.0, sample_rate, ((2*np.arctan(140*duree))/(np.pi))))

```



polyphony and analog synthesis

```
66 def simultane2(note1,t1,note2,t2):
67     if t1 == t2 :
68         somme = (karplus_strong(note1,t1+1.0,sample_rate,((2*np.arctan(140*t1))/(np.pi))))+(karplus_strong(note2,t2+1.0,sample_rate,((2*np.arctan(140*t2))))
69         print(somme)
70         plt.plot(somme)
71         plt.title('somme')
72         plt.grid()
73         plt.xlabel('échantillons')
74         plt.ylabel('amplitude')
75         plt.show()
76         #write("/Users/julienfauvel/Downloads/superposition2.wav", sample_rate, somme)
77         sd.play(somme)
78         sd.wait()
79         return(somme)
80     else :
81         if t1 < t2 :
82             L = (karplus_strong(note1,t2+1.0,sample_rate,((2*np.arctan(140*t1))/(np.pi))))
83             somme2bis = L+(karplus_strong(note2,t2+1.0,sample_rate,((2*np.arctan(140*t2))))
84             plt.plot(somme2bis)
85             plt.title('t1<t2')
86             plt.show()
87             write("/Users/julienfauvel/Downloads/Karplus_A_2_Note_t1<T2.wav", sample_rate, somme2bis)
88             return (somme2bis)
89         else :
90             t1,t2=t2,t1
91             L = (karplus_strong(note1,t2+1.0,sample_rate,((2*np.arctan(140*t1))/(np.pi))))
92             somme2bis = L+(karplus_strong(note2,t2+1.0,sample_rate,((2*np.arctan(140*t2))))
93             plt.plot(somme2bis)
94             plt.title('t1<t2')
95             plt.show()
96             write("/Users/julienfauvel/Downloads/Karplus_A_2_Note_t1<T2.wav", sample_rate, somme2bis)
97             return (somme2bis)
```

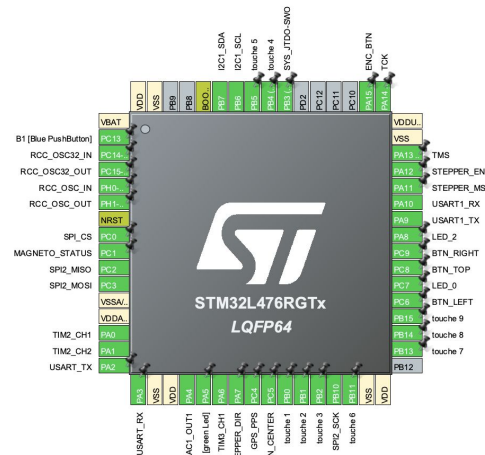


The white noise generator can be made using analog methods. The filter and delay as well.
Pros : not using a DAC

Implémentation Logicielle en C

Changements lors du passage en C :

- HAL
- Configuration du DAC
- Tim2=16kHz
- Configuration des Gpio pour les touches
- Enveloppe ADSR



```
karplus_strong.h | adsr.h | adsr.c | karplus_strong.c | main.c | MX
1  /*
2   * adsr.c
3   *
4   * Created on: May 18, 2026
5   * Author: widad
6   */
7
8
9  #include "adsr.h"
10
11 void ADSR_Init(ADSR_t *env, float a, float d, float s, float r)
12 {
13     env->attack = a;
14     env->decay = d;
15     env->sustain = s;
16     env->release = r;
17
18     env->level = 0.0f;
19     env->state = ADSR_OFF;
20 }
21
22 void ADSR_NoteOn(ADSR_t *env)
23 {
24     env->state = ADSR_ATTACK;
25 }
26
27 void ADSR_NoteOff(ADSR_t *env)
28 {
29     env->state = ADSR_RELEASE;
30 }
31
32 float ADSR_Step(ADSR_t *env)
33 {
34     switch(env->state)
35     {
36         case ADSR_ATTACK:
37             env->level += env->attack;
38             if (env->level >= 1.0f) {
39                 env->level = 1.0f;
40                 env->state = ADSR_DECAY;
41             }
42             break;
43
44         case ADSR_DECAY:
45             env->level -= env->decay;
46             if (env->level <= env->sustain) {
47                 env->level = env->sustain;
48                 env->state = ADSR_SUSTAIN;
49             }
50             break;
51
52         case ADSR_SUSTAIN:
53             // niveau constant
54             break;
55
56         case ADSR_RELEASE:
57             env->level -= env->release;
58             if (env->level <= 0.0f) {
59                 env->level = 0.0f;
60                 env->state = ADSR_OFF;
61             }
62             break;
63
64         case ADSR_OFF:
65             default:
66                 env->level = 0.0f;
67                 break;
68     }
69
70     return env->level;
71 }
72
```

Implémentation Logicielle en C

Test sur PC :

Création d'un projet jumeau mais sans la fonction HAL

pour configurer sur le terminal du PC

AUDIO à faire
écouter

```
*main_pc.c X
56
57 int main()
58 {
59     KarplusStrong_t ks;
60     ADSR_t env;
61
62     ADSR_Init(&env, 0.002f, 0.01f, 0.4f, 0.005f);
63     // KS_Init(&ks, 440.0f, 0.996f); // note jouer dans le test
64
65     // // pls notes a la suite
66     // float notes[] = {261.63f, 293.66f, 329.63f, 349.23f}; // Do Ré Mi Fa
67     // int nb_notes = 4;
68     //
69     // int pos = 0;
70     // float buffer_f[SAMPLE_RATE * 10]; // 10 sec max
71     //
72     // for (int n = 0; n < nb_notes; n++) {
73     //
74     //     // Réinitialiser l'ADSR et le KS pour chaque note
75     //     ADSR_Init(&env, 0.01f, 0.2f, 0.8f, 0.3f);
76     //     KS_Init(&ks, notes[n], 0.996f);
77     //
78     //     ADSR_NoteOn(&env);
79     //
80     //     // Durée de la note (0.5 sec)
81     //     for (int i = 0; i < SAMPLE_RATE * 0.5; i++) {
82     //         float sample = KS_Step(&ks) * ADSR_Step(&env);
83     //         buffer_f[pos++] = sample;
84     //     }
85     //
86     //     // Lâcher la note
87     //     ADSR_NoteOff(&env);
88     //
89     //     // Laisser le release se terminer (0.2 sec)
90     //     for (int i = 0; i < SAMPLE_RATE * 0.2; i++) {
91     //         float sample = KS_Step(&ks) * ADSR_Step(&env);
92     //         buffer_f[pos++] = sample;
93     //     }
94     // }
95
96 }
```

```

96 //accord de Do majeur (do mi sol)
97
98 float notes[] = {261.63f, 329.63f, 392.00f};
99 int nb_notes = 3;
100
101 // Instances multiples
102 KarplusStrong_t ks_accord[3];
103 ADSR_t envs[3];
104
105 int pos = 0;
106 float buffer_f[SAMPLE_RATE * 10];
107
108 // Initialisation des 3 notes
109 for (int n = 0; n < nb_notes; n++) {
110     ADSR_Init(&envs[n], 0.01f, 0.2f, 0.8f, 0.3f);
111     KS_Init(&ks_accord[n], notes[n], 0.996f);
112     ADSR_NoteOn(&envs[n]);
113 }
114
115 // Durée de l'accord : 1 seconde
116 for (int i = 0; i < SAMPLE_RATE * 1.0f; i++) {
117
118     float mix = 0.0f;
119
120     for (int n = 0; n < nb_notes; n++) {
121         float s = KS_Step(&ks_accord[n]) * ADSR_Step(&envs[n]);
122         mix += s;
123     }
124
125     mix /= nb_notes; // normalisation
126     buffer_f[pos++] = mix;
127 }
128
129 // Release des 3 notes
130 for (int n = 0; n < nb_notes; n++) {
131     ADSR_NoteOff(&envs[n]);
132 }
133
134 for (int i = 0; i < SAMPLE_RATE * 0.3f; i++) {
135
136     float mix = 0.0f;
137
138     for (int n = 0; n < nb_notes; n++) {
139         float s = KS_Step(&ks_accord[n]) * ADSR_Step(&envs[n]);
140         mix += s;
141     }
142
143     mix /= nb_notes;
144     buffer_f[pos++] = mix;
145 }
146
147 int total_samples = pos;
148 int16_t *buffer = malloc(total_samples * sizeof(int16_t));
149
150 for (int i = 0; i < total_samples; i++) {
151     float s = buffer_f[i];
152     if (s > 1.0f) s = 1.0f;
153     if (s < -1.0f) s = -1.0f;
154     buffer[i] = (int16_t)(s * 32767.0f);
155 }
156
157 write_wav("test.wav", buffer, total_samples);
158 free(buffer);
159
160 return 0;
161
162
163
164
165 }
```

Hardware Assembly



SMT Reflow Soldering: Solder paste application via stencil for surface-mount components (AMS1117 regulator, Amplifier, USB), followed by a pass through the thermal oven.



Manual THT Soldering: Traditional iron soldering for all through-hole components (Cherry MX switches, electrolytic capacitors, Jack, potentiometer).

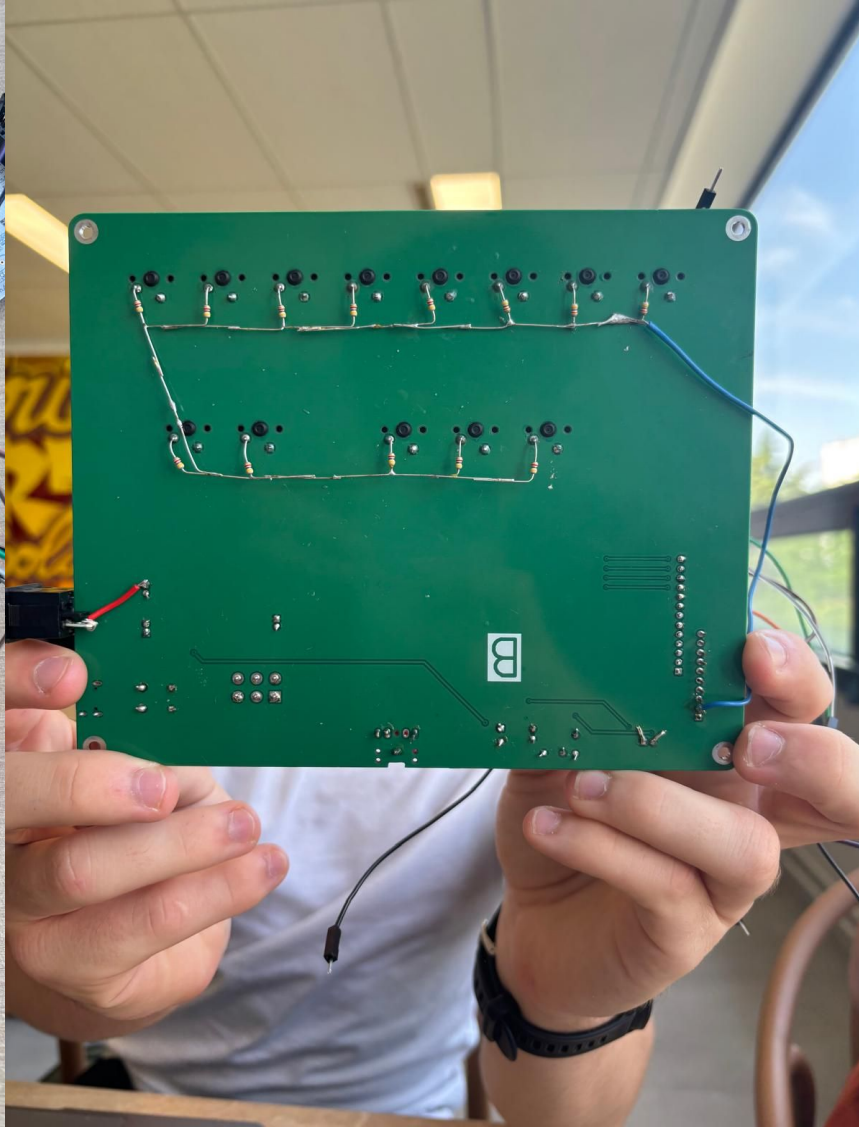
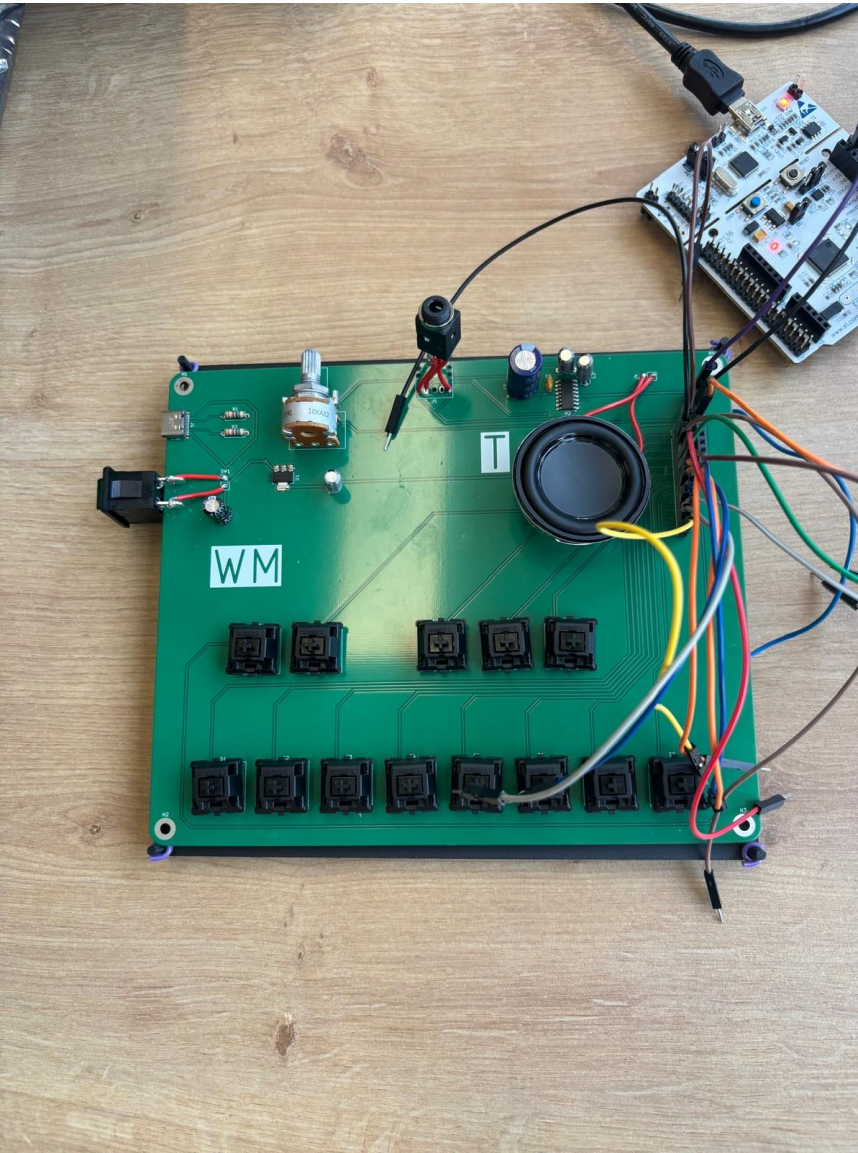


Troubleshooting: Missing footprints for pull-up resistors (10k Ω) on the original layout. We bypassed the issue by manually soldering them to the back of the board, directly onto the switch pins.

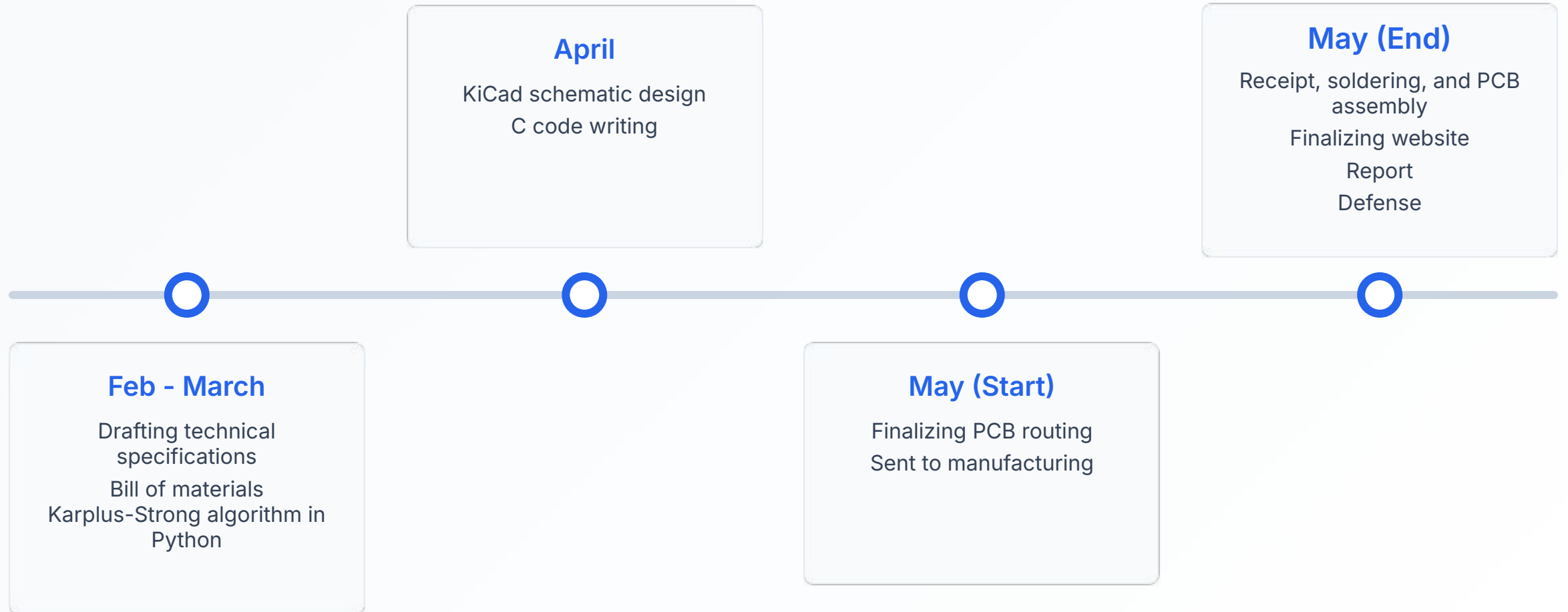


Testing: Oscilloscope verification of GPIO signals, the correct Timer frequency (44.1kHz), and clean 3.3V voltage before connecting the Nucleo.

Soldering and 3D Printing



Time Management



Conclusion & Outlook

CSR Approach

Our project integrates environmental considerations:

- **Repairability:** Use of many THT components (potentiometer, jack, switches) that can be individually replaced with a soldering iron.
- **Durability:** Switches guaranteed for 50 million keystrokes.
- **Modularity:** Our circuit fits onto the Nucleo board without permanent soldering, so we can reuse the Nucleo board for other projects in the future.

Future Improvements

The current prototype provides a solid basis for evolution:

- **Hardware:** Adding an I2C multiplexer to exceed the limit of GPIO pins and manage a full 4-octave keyboard.
- **Sound:** Improve the code so that the note resonates exactly like a real piano.
- **Wireless:** Add Bluetooth to connect it directly to a computer.

Merci de votre attention

Retrouvez notre projet sur :



m9kmdn7245-gif.github.io/2526--P1-piano-electronique

Widad • Élie • Julien • Léo • Wissem • Claire

ENSEA – Projet Électronique 1A

